
THE KNOWLEDGE LIFECYCLE OF LARGE LANGUAGE MODELS

 **Amey Thakur**
Independent Researcher
Toronto, Canada
ameythakur20@gmail.com

 **Sarvesh Talele**
Independent Researcher
Mumbai, India
talelesarvesh@gmail.com

August 17, 2026

ABSTRACT

The emergence of reasoning models and inference-time scaling has altered how large language models (LLMs) process knowledge. Earlier systems relied on static pipelines built from retrieval-augmented generation (RAG), knowledge editing, and continual learning; modern systems resolve conflicts during test-time compute. This shift exposes a methodological gap: there is currently no unified formalism for how knowledge persists, evolves, and is computed over across a model’s lifespan. This paper introduces the **Knowledge Lifecycle**, a framework that organizes LLM knowledge management into five structural stages: *Acquisition*, *Storage*, *Retrieval*, *Update*, and *Forgetting*. We make three contributions. First, we formalize the five stages and their cross-stage failure modes, showing that the fragmented subfields addressing each stage in isolation leave the boundaries between stages unstudied. Second, we propose the **Provenance Vector** (p), an architectural modification that stores temporal and source metadata alongside parametric memories, enabling a gating mechanism that suppresses stale parametric activations during test-time conflict resolution. Third, we introduce **Lifecycle Desynchronization** ($\mathcal{D}_{sync}$), a metric based on Kullback-Leibler (KL) divergence that quantifies the predictive failure occurring when parametric weights and non-parametric context conflict. In a reproducible probe of a documented pharmaceutical withdrawal, GPT-2 assigns the correct continuation a probability of 5.9×10^{-6} even when the withdrawal notice is present in context ($\mathcal{D}_{sync} = 12.05$ nats), while the context shifts the model’s full output distribution by only 0.03 nats. These measurements demonstrate near-total parametric dominance and motivate lifecycle-aware architectures for reliable deployment.

List of Abbreviations

LLM	Large Language Model
RAG	Retrieval-Augmented Generation
KL	Kullback-Leibler (divergence)
RLHF	Reinforcement Learning from Human Feedback
RepE	Representation Engineering
FFN	Feed-Forward Network
VLM	Vision-Language Model
FDA	Food and Drug Administration
MIPS	Maximum Inner Product Search
CE	Cross-Entropy
MSE	Mean Squared Error
DPR	Dense Passage Retrieval
ROME	Rank-One Model Editing

MEMIT	Mass-Editing Memory in Transformers
LoRA	Low-Rank Adaptation
EWC	Elastic Weight Consolidation

1 Introduction

Large language models serve as the backbone of systems that answer medical questions, draft legal briefs, write production code, and tutor students in mathematics (Brown et al., 2020; Achiam et al., 2023; Gemini Team, Google, 2023). With the advent of inference-time scaling and reasoning models such as OpenAI o1 and DeepSeek-R1 (OpenAI, 2024; DeepSeek-AI, 2025), these systems no longer simply retrieve and generate; they execute extended test-time compute to search, reflect, and self-correct. This advanced reasoning, however, is bound by a specific bottleneck: the model’s underlying relationship with facts. A medical reasoning agent that computes over a retracted drug interaction, or a coding agent that builds logic around a deprecated API, represents a failure of knowledge management rather than a failure of reasoning capacity.

Current responses to these challenges are fragmented. General surveys on LLMs (Zhao et al., 2023; Wang et al., 2024) touch on knowledge tangentially, while specific subfields operate in isolation. At least five separate subfields now address different pieces of the knowledge problem:

1. **Retrieval-Augmented Generation (RAG)** focuses on how to fetch relevant documents and feed them to the model at inference time (Lewis et al., 2020; Guu et al., 2020; Gao et al., 2023).
2. **Knowledge Editing** asks whether individual facts inside a model’s weights can be surgically modified without retraining (Meng et al., 2022, 2023; Yao et al., 2023).
3. **Continual Learning** tackles how a model can absorb new information over time without destroying what it already knows (Kirkpatrick et al., 2017; Ke et al., 2023; Wu et al., 2024).
4. **Context Engineering** studies how to arrange, compress, and prioritize information within a finite context window (Liu et al., 2024).
5. **Agent Memory** builds persistent storage that lets LLM-based agents remember and reuse past experience (Park et al., 2023; Packer et al., 2023; Hu et al., 2025).

Each subfield has its own dedicated surveys (Gao et al., 2023; Mialon et al., 2023; Hu et al., 2025; Wu et al., 2025). They rarely integrate their methodologies. Optimizing a RAG retriever seldom includes an evaluation of what happens when retrieved facts contradict the model’s parametric memory. A team building a knowledge editor rarely checks whether their edits survive continual fine-tuning. These isolated evaluations obscure cross-stage system failures.

This paper proposes an integrated approach. Rather than treating these problems in isolation, we formalize them as stages of a single unified architecture: the **Knowledge Lifecycle** of large language models. The lifecycle defines five structural stages: how knowledge is *acquired* and *stored* (§4), how it is *retrieved* when needed (§4), how it is *updated* as the world changes (§5), and how it is *forgotten*, whether deliberately or by accident (§6).

Contributions. We make three contributions:

1. **A lifecycle framework** that unifies existing research across five structural stages (§3–§7). We formalize the operational mechanics of each stage and map twenty representative works onto the stages they address, showing that the boundaries between stages are where the least-studied and most damaging failures occur.
2. **The Provenance Vector** (p) (§8), an architectural proposal that stores temporal and source metadata alongside each parametric memory. A gating function reads this metadata during inference and suppresses stale activations, allowing retrieved context to dominate without destructive weight edits. We give the forward pass as an explicit algorithm, derive the parameter overhead, and prove an idealized consistency result with clearly stated assumptions.
3. **Lifecycle Desynchronization** (D_{sync}) (§8), an evaluation metric that measures the failure occurring when model weights are out of sync with external retrieval indices. We define the metric, give its measurement protocol as an algorithm, and report a reproducible probe on GPT-2 in which the correct continuation receives probability 5.9×10^{-6} despite the corrective document being present in context.

To our knowledge, no prior work has formalized the knowledge management problem across all five lifecycle stages.

Table 1: Coverage of lifecycle stages in prior literature. ✓ = primary focus; ~ = partial; – = absent.

Work	Acquire	Store	Retrieve	Update	Forget
Gao et al. (2023), on RAG	–	~	✓	–	–
Mialon et al. (2023), Augmented LMs	~	–	✓	–	–
Yao et al. (2023), Knowledge Editing	–	~	–	✓	–
Wu et al. (2024), Continual Learning	~	–	–	✓	~
Hu et al. (2025), Agent Memory	–	✓	✓	~	~
Wu et al. (2025), Human→AI Memory	~	✓	~	~	–
Our framework	✓	✓	✓	✓	✓

2 Background and Scope

2.1 What Counts as “Knowledge” in an LLM?

We use “knowledge” broadly. Within the Knowledge Lifecycle framework, knowledge is any information, whether factual, procedural, or experiential, that shapes a model’s outputs. Four varieties matter here:

- **Factual knowledge.** Concrete facts about the world (“The boiling point of water at sea level is 100°C”), acquired mainly during pre-training (Petroni et al., 2019).
- **Procedural knowledge.** How to carry out tasks: following multi-step instructions, writing code in a particular style, or reasoning through a chain of logic. This is typically instilled through instruction tuning and Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022; Wei et al., 2022).
- **Experiential knowledge.** Records of past interactions, user preferences, and task outcomes kept by agent memory systems (Park et al., 2023; Packer et al., 2023).
- **Contextual knowledge.** Information provided at inference time via the prompt or fetched from an external source (Lewis et al., 2020).

One important observation: the same fact can exist in multiple forms simultaneously. “The CEO of OpenAI is Sam Altman” might live in the model’s weights (parametric), in a vector database (non-parametric), and in the current prompt (contextual). Each copy follows a different lifecycle path, with different update and deletion properties.

2.2 Relationship to Existing Literature

Table 1 shows where prior literature sits. Each subfield covers one or two lifecycle stages in depth; none attempts the full picture.

Three works require explicit differentiation. Wu et al. (2025) drew a productive analogy between human memory systems (sensory, working, long-term) and LLM memory architectures. Their contribution is taxonomic: it classifies memory types but does not formalize the transitions between them or identify the failure modes that arise when stages interact. Our framework differs in that it treats the stages as an *operational architecture* with mathematically defined interfaces, not as a classification scheme.

Hu et al. (2025) provided the most comprehensive survey of agent memory to date, covering storage hierarchies, retrieval scoring functions, and consolidation mechanisms. Their scope, however, is restricted to agent systems; they do not address parametric storage, knowledge editing, or machine unlearning. Our framework subsumes agent memory as a special case, one in which all five stages operate within a single persistent system, and extends the analysis to the broader LLM ecosystem.

Gao et al. (2023) surveyed retrieval-augmented generation in depth but treated retrieval as a self-contained pipeline. Their analysis does not consider what happens when retrieved facts contradict the model’s parametric memory, or when a retrieved fact has been marked for deletion under privacy regulations. These are precisely the cross-stage failure modes that our lifecycle analysis is designed to expose.

2.3 Scope and Boundaries

We restrict attention to text-based LLMs built on the Transformer architecture (Vaswani et al., 2017). Multimodal extensions appear only in our discussion of future work (§9). When we discuss agent memory, we treat the LLM as a component inside a larger agent system and make the boundary explicit.

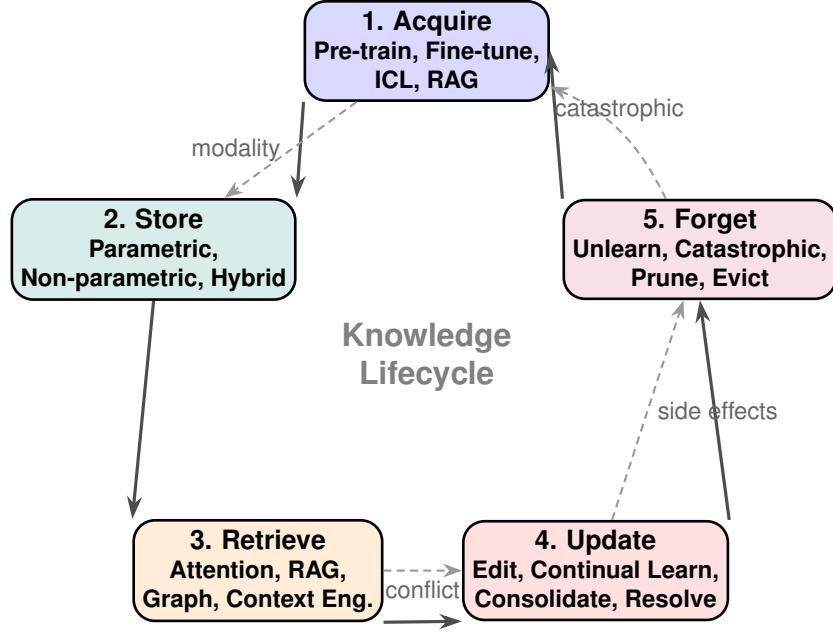


Figure 1: The Knowledge Lifecycle framework. Solid arrows show the primary flow of knowledge through five stages. Dashed arrows indicate cross-stage interactions that produce failure modes (knowledge conflicts, catastrophic forgetting, unintended side effects). Each stage maps to a distinct body of research, but the interactions between stages are where the hardest open problems lie.

3 The Knowledge Lifecycle Framework

The lifecycle has five stages. Each one maps to a well-defined set of research questions. Figure 1 gives the visual overview.

3.1 Overview of the Five Stages

Stage 1: Acquisition. Knowledge enters the system. Pre-training compresses billions of web pages into parameters (Kaplan et al., 2020; Hoffmann et al., 2022). Fine-tuning injects specialized task or domain knowledge (Hu et al., 2022). In-context learning supplies knowledge transiently through demonstrations placed in the prompt (Min et al., 2022). Retrieval-augmented pre-training (REALM (Gua et al., 2020), Atlas (Izacard et al., 2023)) teaches the model to pull in external text during training itself.

Stage 2: Storage. Acquired knowledge has to live somewhere. We identify four storage modalities: *parametric* (baked into weights), *non-parametric* (sitting in a vector database, knowledge graph, or document index), *hybrid* (architectures such as RETRO (Borgeaud et al., 2022) and nearest-neighbor language models (Khandelwal et al., 2020) that blend both), and *agent memory hierarchies* (tiered systems with working, episodic, and archival layers (Packer et al., 2023)).

Stage 3: Retrieval. A question arrives. The system must locate the right knowledge and bring it to bear. Internal retrieval works through the self-attention mechanism. External retrieval is the RAG pipeline: encode the query, search the index, return the top passages (Karpukhin et al., 2020); graph-structured variants extend it from passage lookup to corpus-level summarization (Edge et al., 2024). Context engineering, meaning the choice of which passages to include, in what order, and in what format, determines whether retrieved knowledge actually reaches the model’s reasoning process (Liu et al., 2024).

Stage 4: Update. The world changes, and the system’s knowledge must change with it. Knowledge editing rewrites individual facts in the weights (Meng et al., 2022, 2023). Continual learning extends the model to new domains over time (Kirkpatrick et al., 2017). Agent memory consolidation turns raw interaction logs into distilled insights (Park et al., 2023). Knowledge conflict resolution decides what to do when the weights say one thing and the retrieved document says another (Xie et al., 2024; Longpre et al., 2021).

Stage 5: Forgetting. Knowledge leaves the system, on purpose or by accident. Machine unlearning removes specific facts to comply with privacy law (Jang et al., 2023; Maini et al., 2024). Catastrophic forgetting destroys old knowledge as a side effect of learning new knowledge (Kirkpatrick et al., 2017; Luo et al., 2023). Selective pruning trims bloated agent memory stores. KV-cache eviction drops tokens that no longer fit in the context window (Xiao et al., 2024).

3.2 Cross-Stage Dependencies

The five stages are not a linear pipeline. They interact, and those interactions create problems that no single stage can solve alone:

- **Acquisition** $\leftrightarrow$ **Storage.** How knowledge is acquired dictates where it ends up. Pre-training stores knowledge parametrically; RAG stores it externally. This choice echoes through every later stage.
- **Storage** $\leftrightarrow$ **Retrieval.** Parametric storage relies on implicit retrieval through attention. Non-parametric storage requires an explicit search pipeline. The retrieval mechanism must match the storage format.
- **Retrieval** $\leftrightarrow$ **Update.** Editing a fact in the weights does not edit it in the vector database, and vice versa. When the two copies disagree, the system produces contradictory outputs.
- **Update** $\leftrightarrow$ **Forgetting.** Every edit is a small forgetting event: the old fact is overwritten. Every unlearning operation risks collateral damage to neighboring facts. The two problems share root causes.
- **Forgetting** $\leftrightarrow$ **Acquisition.** Catastrophic forgetting during continual training and deliberate unlearning for privacy are studied by different communities, but both reduce to the same question: how to selectively remove information from entangled representations.

4 Formalizing the Legacy Pipeline

Before examining how inference-time scaling alters the lifecycle, we formally define the constraints of the Acquisition, Storage, and Retrieval stages. These stages established the baseline mathematics of LLM knowledge management prior to the advent of reasoning models.

4.1 Acquisition and Parametric Storage

Pre-training and fine-tuning serve as the primary mechanisms for eager knowledge acquisition. Unlike adversarial generative training, in which a discriminator supplies the learning signal (Thakur and Satish, 2021b), autoregressive language models learn by maximum likelihood: the model compresses factual, linguistic, and procedural information into its weights by minimizing the cross-entropy loss over a training corpus $\mathcal{D}$:

$$\mathcal{L}_{\text{CE}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}) \right] \quad (1)$$

Intuition. This is a mimicry objective. The model predicts the next token in a sequence and receives a penalty when its prediction deviates from the ground truth. The process forces factual knowledge into the parameters θ , but it does so without any metadata: the model learns *what* is true, but not *when* or *why* it is true. This absence of provenance is the root cause of the failures analyzed in §8.

Once acquired, this knowledge resides implicitly within the feed-forward networks (FFNs), the classical dense building blocks of neural computation (Thakur and Konde, 2021). Geva et al. (2021) established that Transformer FFNs operate as associative key-value memories. For an input representation x , the FFN computation is:

$$\text{FFN}(x) = f(xK^{\top})V = \sum_{i=1}^{d_m} f(x \cdot k_i) v_i \quad (2)$$

where k_i acts as a key detecting a specific semantic pattern, and v_i acts as a value inducing a corresponding distribution over the output vocabulary.

Intuition. The FFN behaves as a large, fuzzy look-up table. The keys k_i are concepts the model has memorized, and the values v_i are the answers associated with those concepts. When the input x matches a key (high dot product $x \cdot k_i$), the model recalls the corresponding answer. The difficulty is that these keys and values are entangled across billions of parameters, which makes it nearly impossible to surgically delete or update a single fact without affecting others (Yao et al., 2023).

4.2 Retrieval-Augmented Constraints

To bypass the rigidity of parametric storage, the field adopted Retrieval-Augmented Generation (Lewis et al., 2020). RAG decouples knowledge storage from the model parameters by maintaining a non-parametric external index. Retrieval is typically framed as a Maximum Inner Product Search (MIPS) over a bi-encoder space (Karpukhin et al., 2020):

$$d^* = \arg \max_{d \in \mathcal{C}} (E_Q(q)^\top E_D(d)) \quad (3)$$

External retrieval solves the immutability problem of parametric weights, but it introduces a critical bottleneck. When the retriever injects documents into the context window, the knowledge must pass through the model’s self-attention mechanism. If the retrieved context contradicts the model’s highly confident parametric weights, standard forward-pass decoding lacks a formal mechanism to resolve the conflict, and the outcome is unpredictable (Xie et al., 2024; Longpre et al., 2021). Our measurements in §8.5 quantify exactly this failure.

5 Update: From Static Editing to Test-Time Resolution

5.1 Static Parameter Modification and Its Limits

The field first approached knowledge updating through static parameter modification. Early work framed editing as learned constrained optimization over the weights (De Cao et al., 2021); ROME (Meng et al., 2022) and MEMIT (Meng et al., 2023) rewrite specific factual associations in the FFN weights through rank-one and mass edits; MEND (Mitchell et al., 2022a) learns an editing network; SERAC (Mitchell et al., 2022b) routes around the base model with an external edit memory; Transformer-Patcher (Huang et al., 2023) and PMET (Li et al., 2024) refine the locality of the intervention; task arithmetic composes whole edit directions in weight space (Ilharco et al., 2023). Across this line of work a consistent limitation emerged, known as the specificity bottleneck: altering a single fact perturbs entangled neighbor representations, and edit quality degrades as edits accumulate (Yao et al., 2023).

5.2 Representation Engineering and Activation Steering

An alternative line of work controls behavior without permanently modifying weights. Representation Engineering (RepE) (Zou et al., 2023) monitors and steers internal activations at inference time: a steering vector added to or gated over the residual stream biases generation away from an undesired direction while the base weights remain intact. Applied to the knowledge lifecycle, steering offers a path around the specificity bottleneck. When a conflict between a retrieved document and a parametric memory is detected, the system can suppress the stale parametric activation for the duration of the query instead of destructively editing it.

5.3 Test-Time Compute and Conflict Resolution

Reasoning models trained with reinforcement learning, such as OpenAI o1 (OpenAI, 2024) and DeepSeek-R1 (DeepSeek-AI, 2025), allocate extended test-time compute to search, reflect, and self-correct before answering. In the lifecycle view, this changes where the Update stage can act: conflicts no longer need to be resolved in a single forward pass, because the model can evaluate competing continuations during its reasoning trajectory. The open question is what signal the model should use to score those continuations. Without metadata recording when and where a parametric fact was acquired, even an extended search has no principled basis for preferring a freshly retrieved fact over a confident but outdated parametric one (Xie et al., 2024). Supplying exactly this signal is the purpose of the Provenance Vector (§8).

6 Forgetting

6.1 Machine Unlearning

Machine unlearning attempts to remove specific data points from the weights, often to comply with privacy regulations (Jang et al., 2023). Standard approaches apply gradient ascent on the forget-set or fine-tune on obfuscated targets (Eldan and Russinovich, 2023). Evaluations on benchmarks such as TOFU (Maini et al., 2024) consistently demonstrate the same entanglement problem seen in editing: aggressive unlearning inflicts collateral damage on retained knowledge, while conservative unlearning leaves the target recoverable through indirect queries. Training-data extraction attacks (Carlini et al., 2021) supply the adversarial standard against which unlearning must be verified.

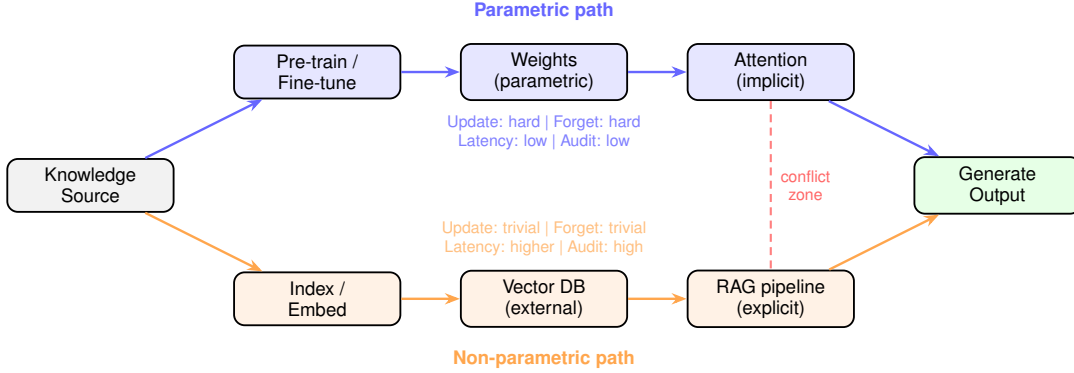


Figure 2: Two paths through the knowledge lifecycle. The parametric path (top, blue) bakes knowledge into weights during training; it is fast at inference but hard to update or forget. The non-parametric path (bottom, orange) stores knowledge externally; it is easy to modify but depends on retrieval quality. When both paths are active, the “conflict zone” (dashed red) produces the knowledge conflict problem discussed in §5.

6.2 Structural Forgetting

The most routine form of forgetting is structural: context window eviction. Despite advances in streaming attention (Xiao et al., 2024) and selective state-space models (Gu and Dao, 2023), active context is bounded, and when an agent’s memory stream exceeds it, tokens are dropped. Agent memory hierarchies such as MemGPT (Packer et al., 2023) mitigate this by paging critical information back into active context, which transforms structural forgetting into a recursive retrieval problem. Catastrophic forgetting during continual training (Kirkpatrick et al., 2017; Luo et al., 2023) completes the picture: knowledge can leave the system as an unintended side effect of acquisition itself.

7 Cross-Cutting Analysis

7.1 Systematic Coverage Analysis of the Literature

To substantiate the claim that existing work is fragmented, we conducted a systematic mapping of representative publications to the lifecycle stages they address. Table 2 presents this analysis. A checkmark indicates that the paper makes a direct methodological or empirical contribution to that stage; a dash indicates no coverage.

The pattern is quantitatively clear. Of the 20 representative works mapped, 5 address only a single lifecycle stage, 10 address exactly two, 3 address three, and only 2 (Generative Agents and MemGPT) touch four or more. The median coverage is 2 stages. No prior work provides a unified formalism across all five. The Update–Forgetting boundary and the Acquisition–Forgetting boundary are the least studied intersections, despite being the source of the most damaging real-world failures, such as fine-tuning that silently overwrites earlier alignment.

7.2 Parametric vs. Non-Parametric: A Unified View

The single most important design decision in any LLM knowledge system is where to put the knowledge: in the weights or outside them. Figure 2 visualizes the two paths, and Table 3 lays out how this choice plays out across every lifecycle stage.

The pattern is consistent. Parametric storage is fast and tightly integrated with the model’s reasoning, but opaque, rigid, and difficult to modify. Non-parametric storage is modular and transparent, but depends entirely on the retrieval pipeline’s ability to surface the right knowledge at the right time. The most promising path forward is lifecycle-aware hybrid design: route stable, frequently accessed knowledge into the weights, and route dynamic, auditable, regulation-sensitive knowledge into external stores where it can be updated and deleted without touching the model.

7.3 Evaluation Methods and Benchmarks

Every lifecycle stage has its own evaluation protocol, and none of them accounts for the others. RAG systems measure retrieval recall and end-to-end answer quality (Es et al., 2024; Chen et al., 2024). Knowledge editing measures efficacy, specificity, and generalization (Yao et al., 2023). Unlearning measures whether the target knowledge can be

Table 2: Literature coverage across lifecycle stages. Each row maps a representative work to the stages it addresses. The rightmost column counts the number of stages covered. The concentration of single- and two-stage papers confirms that the field operates in silos; cross-stage interactions remain largely unstudied.

Representative Work	Acquire	Store	Retrieve	Update	Forget	Stages
Lewis et al. (RAG, 2020)	–	–	✓	–	–	1
Karpukhin et al. (DPR, 2020)	–	–	✓	–	–	1
Meng et al. (ROME, 2022)	–	✓	–	✓	–	2
Meng et al. (MEMIT, 2023)	–	✓	–	✓	–	2
Mitchell et al. (SERAC, 2022)	–	✓	–	✓	–	2
Kirkpatrick et al. (EWC, 2017)	✓	–	–	✓	✓	3
Jang et al. (Knowledge Unlearning, 2023)	–	–	–	–	✓	1
Maini et al. (TOFU, 2024)	–	–	–	–	✓	1
Borgeaud et al. (RETRO, 2022)	✓	✓	✓	–	–	3
Guu et al. (REALM, 2020)	✓	–	✓	–	–	2
Izacard et al. (Atlas, 2023)	✓	–	✓	–	–	2
Park et al. (Generative Agents, 2023)	✓	✓	✓	✓	✓	5
Packer et al. (MemGPT, 2023)	–	✓	✓	✓	✓	4
Xie et al. (Knowledge Conflicts, 2024)	–	✓	✓	✓	–	3
Hu et al. (LoRA, 2022)	✓	✓	–	–	–	2
Kaplan et al. (Scaling Laws, 2020)	✓	–	–	–	–	1
Khandelwal et al. (kNN-LM, 2020)	–	✓	✓	–	–	2
Carlini et al. (Data Extraction, 2021)	–	✓	–	–	✓	2
Luo et al. (Catastrophic Forgetting, 2023)	✓	–	–	–	✓	2
Edge et al. (GraphRAG, 2024)	–	✓	✓	–	–	2
This paper	✓	✓	✓	✓	✓	5

Table 3: How the parametric/non-parametric choice affects each lifecycle stage.

Lifecycle Stage	Parametric	Non-Parametric
Acquisition	Pre-training, fine-tuning	Indexing documents
Storage	Distributed in weights	Explicit in databases
Retrieval	Attention (implicit)	Search pipeline (explicit)
Update	Knowledge editing (hard)	Database write (trivial)
Forgetting	Unlearning (hard)	Delete entry (trivial)
Latency	Low (single forward pass)	Higher (retrieval + generation)
Auditability	Low (opaque weights)	High (traceable sources)
Scalability	Fixed at training time	Grows with corpus

extracted through membership inference and extraction attacks (Maini et al., 2024; Carlini et al., 2021). Agent memory systems are beginning to use multi-session benchmarks that test whether an agent can recall information from many conversations ago (Hu et al., 2025).

The gap is structural. A system can score well on a RAG benchmark and a knowledge editing benchmark individually, while failing when the two stages interact, for example when an edited fact is overridden by a stale retrieved passage. What the field needs is a lifecycle benchmark: a test suite that evaluates a sequence of operations (store, retrieve, update, verify the update, forget, verify the deletion) on the same set of facts. No such benchmark exists today.

7.4 The Knowledge Consistency Problem

The hardest cross-stage problem is consistency. As knowledge flows through the lifecycle, inconsistencies accumulate from at least four sources:

- **Temporal.** The weights reflect the training cutoff. The retrieved documents reflect the current date. The model may blend the two, producing an answer that is partly outdated and partly current.
- **Source-level.** Two retrieved documents may contradict each other. The model has no mechanism to determine which source is more reliable.
- **Edit-level.** An edit changes “The capital of Country X is A” to “The capital of Country X is B,” but leaves all related facts (“A is the largest city in Country X”) unchanged. The model’s knowledge is now internally contradictory.

- **Forgetting-level.** An unlearning operation removes a fact when asked directly, but the model can still reconstruct it from indirect cues. The fact is technically forgotten but practically recoverable.

Consistency is a lifecycle problem by definition. It cannot be fixed at any single stage. Fixing it requires a provenance system that tracks every fact through every stage: when it was acquired, where it is stored, how it has been retrieved, whether it has been updated, and whether it has been marked for deletion. The next section develops the architectural and metrological components of exactly such a system.

8 Original Contributions

Based on the five-stage analysis, we propose two original technical components designed to address the fundamental asymmetries of the knowledge lifecycle. Table 4 summarizes the failure modes identified in the preceding sections and the corresponding mechanisms introduced here.

Table 4: Synthesis of Knowledge Lifecycle failure modes and the mechanisms proposed in this work.

Stage	Primary Failure Mode	Root Cause	Proposed Mechanism
Acquisition	Entanglement	Metadata-free compression	Provenance Vector (p)
Storage	Boundary conflict	Parametric/non-parametric gap	Metric: $\mathcal{D}_{sync}$
Retrieval	Context override failure	Static forward-pass priority	Provenance-aware scoring
Update	Specificity bottleneck	Destructive weight modification	Activation gating
Forgetting	Collateral damage	Gradient-based unlearning	Gate-level suppression

8.1 Architectural Proposal: The Provenance Vector (p)

Transformers store facts implicitly as key-value pairs in FFNs (§4.1). When knowledge conflicts arise (§5), the network has no mathematical basis for prioritizing one fact over another, because provenance, meaning when and from where the fact was acquired, is discarded during parametric compression.

We propose extending each FFN memory slot from a pair (k_i, v_i) to a triple (k_i, v_i, p_i) , where the *Provenance Vector* $p_i \in \mathbb{R}^{d_p}$ is a learned embedding of the acquisition timestamp τ_i and a source reliability score s_i :

$$p_i = E_p([\tau_i; s_i]), \quad E_p : \mathbb{R}^2 \rightarrow \mathbb{R}^{d_p} \quad (4)$$

The provenance store is metadata: it does not enter the residual stream and therefore leaves the dimensionality and function of the standard FFN untouched. Its sole consumer is a gating function evaluated during inference (§8.2).

Training objective. During pre-training, an auxiliary loss trains the provenance store. Given a training example drawn from source s at timestamp τ , the model minimizes a joint objective:

$$\mathcal{L}_{total} = \mathcal{L}_{CE}(\theta) + \alpha \cdot \mathcal{L}_{prov} \quad (5)$$

where $\mathcal{L}_{prov}$ is a Mean Squared Error (MSE) regression loss between a decoding of the provenance embeddings of the activated slots and the ground-truth metadata $[\tau; s]$ of the example, and α balances the two objectives. Slots that activate strongly on an example are pushed to carry that example’s provenance; slots activated by many temporally scattered examples learn an averaged timestamp with correspondingly lower reliability, which is itself informative.

Complexity. The modification adds d_p dimensions to each of the d_m slots per FFN layer. Setting $d_p = 8$ on a 32-layer model with $d_m = 4096$ adds $8 \times 4096 \times 32 \approx 1.05\text{M}$ parameters, below 0.02% of a 7B-parameter model. Inference cost grows by one elementwise gate per FFN slot, which is negligible relative to the matrix products in Equation 2.

8.2 Provenance-Gated Inference

At inference time, let T_{now} be the current timestamp and $\Delta\tau_i = T_{now} - \tau_i \geq 0$ the age of slot i . We define a gate $S(\Delta\tau_i) \in (0, 1)$ over each slot, using a sigmoidal soft threshold; soft membership functions of this kind have long been used to blend symbolic thresholds with continuous activations in neuro-fuzzy systems (Thakur et al., 2021a):

$$S(\Delta\tau_i) = \sigma(\gamma - \beta \cdot \Delta\tau_i) = \frac{1}{1 + \exp(\beta \cdot \Delta\tau_i - \gamma)} \quad (6)$$

Algorithm 1 Provenance-gated FFN forward pass

Require: input representation x ; slots $\{(k_i, v_i, p_i)\}_{i=1}^{d_m}$; current time T_{now} ; sensitivity β ; threshold γ

Ensure: steered FFN output o

```

1:  $o \leftarrow 0$ 
2: for  $i = 1$  to  $d_m$  do
3:    $a_i \leftarrow f(x \cdot k_i)$  ▷ standard key activation
4:   if  $a_i \neq 0$  then
5:      $(\tau_i, s_i) \leftarrow \text{decode}(p_i)$  ▷ read metadata, not the residual stream
6:      $\Delta\tau_i \leftarrow T_{now} - \tau_i$ 
7:      $g_i \leftarrow \sigma(\gamma - \beta \cdot \Delta\tau_i)$  ▷ Equation 6
8:      $o \leftarrow o + g_i \cdot a_i \cdot v_i$ 
9:   end if
10: end for
11: return  $o$ 
    
```

where $\beta > 0$ is a learned sensitivity parameter and $\gamma > 0$ is a freshness threshold. The gate decreases monotonically with age: fresh slots ($\Delta\tau_i \approx 0$) pass with $S \approx \sigma(\gamma) \approx 1$, while stale slots are attenuated. The steered FFN computation becomes:

$$\text{FFN}_{\text{steered}}(x) = \sum_{i=1}^{d_m} S(\Delta\tau_i) f(x \cdot k_i) v_i \quad (7)$$

Adaptive thresholding. The threshold γ need not be static. A reasoning model can compute it during its reflection loop from the entropy of its own predictive distribution: when the model detects a conflict between context and parametric prediction (high entropy or divergent candidate continuations), it lowers γ , making the gate more aggressive for that query. This couples the gate to test-time compute: more deliberation buys sharper conflict resolution, without any weight modification.

Intuition. The gate is a volume control attached to every fact in the model’s memory. A fact’s age turns its volume down; the query’s difficulty decides how strict the ceiling is. An outdated fact that contradicts a fresh retrieved document is attenuated at its source, inside the FFN sum, before it can pollute the residual stream. The base weights are never modified, so there is no collateral damage of the kind that limits static editing (§5).

8.3 Idealized Consistency Analysis

We now state the conditions under which provenance gating provably resolves a parametric-contextual conflict. The result is an idealized analysis, not a guarantee about production systems; its purpose is to show that the mechanism is structurally sound, and its assumptions make explicit what a real implementation must approximate.

Assumption 1. For query q , every FFN slot whose value vector contributes to the outdated answer has age $\Delta\tau_i > \gamma/\beta$, and every slot contributing to other necessary computation (syntax, style, task format) has age $\Delta\tau_i < \gamma/\beta$.

Assumption 2. The retrieved context $\mathcal{M}_{now}$ contains the updated fact, and with the conflicting parametric contribution suppressed, the model’s output distribution on q is determined by attention over $\mathcal{M}_{now}$; denote this distribution $P_{ctx}(y | q, \mathcal{M}_{now})$.

Proposition 1. Under Assumptions 1 and 2, as $\beta \rightarrow \infty$ the gated model’s output distribution converges to P_{ctx} , and consequently

$$\lim_{\beta \rightarrow \infty} D_{KL}(P_{ctx} \| P_{\text{steered}}) = 0. \quad (8)$$

Proof. For any slot with $\Delta\tau_i > \gamma/\beta$, the gate argument $\gamma - \beta\Delta\tau_i \rightarrow -\infty$ as $\beta \rightarrow \infty$, so $S(\Delta\tau_i) \rightarrow \sigma(-\infty) = 0$; for any slot with $\Delta\tau_i < \gamma/\beta$, the argument tends to $+\infty$ and $S \rightarrow 1$. By Assumption 1, exactly the slots supporting the outdated answer are suppressed in Equation 7 while all others pass unchanged. By Assumption 2, the resulting output distribution is P_{ctx} , and $D_{KL}(P_{ctx} \| P_{\text{steered}}) = 0$. $\square$

The assumptions are strong, and stating them is the point. Assumption 1 requires provenance to separate fact slots from infrastructure slots by age, which real training will achieve only approximately; Assumption 2 requires the retriever to have found the updated fact at all. What the proposition establishes is that, given accurate provenance, conflict

Algorithm 2 $\mathcal{D}_{sync}$ measurement protocol

Require: model with weights θ_{stale} and token-probability access; conflict pair (query q , correct continuation y^* , corrective document m)

Ensure: $\hat{\mathcal{D}}_{sync}$, context influence $\mathcal{I}_{ctx}$

- 1: $P_0 \leftarrow \text{softmax}(\text{logits}(q))$ ▷ condition A: no context
 - 2: $P_1 \leftarrow \text{softmax}(\text{logits}(m \oplus q))$ ▷ condition B: corrective context prepended
 - 3: verify tokenization: y^* must map to a defined token sequence in both conditions
 - 4: $\hat{\mathcal{D}}_{sync} \leftarrow -\ln P_1(y^*)$ ▷ Equation 10
 - 5: $\mathcal{I}_{ctx} \leftarrow \sum_y P_1(y) \ln(P_1(y)/P_0(y))$ ▷ Equation 11
 - 6: **return** $\hat{\mathcal{D}}_{sync}$, $\mathcal{I}_{ctx}$, and the top- k tokens of P_0 , P_1 for inspection
-

resolution reduces to a per-slot gating operation with no weight modification, in contrast to static editing where every update risks the specificity bottleneck.

8.4 Evaluation Metric: Lifecycle Desynchronization ($\mathcal{D}_{sync}$)

The field currently evaluates lifecycle stages in isolation, which masks cross-stage failures (§7). We propose a unified metric, *Lifecycle Desynchronization*, that quantifies the failure occurring when a model’s parametric state contradicts its non-parametric state.

Definition. Let θ_{stale} be the deployed model’s weights, $\mathcal{M}_{now}$ the current external knowledge store, and $P_{sync}(y | q)$ the output distribution of a hypothetical fully synchronized system whose weights agree with $\mathcal{M}_{now}$. For a query q probing a fact that changed after the training cutoff:

$$\mathcal{D}_{sync}(q) = D_{KL}(P_{sync}(y | q) \parallel P(y | q, \theta_{stale}, \mathcal{M}_{now})) \quad (9)$$

A large value means the deployed system, even with the corrective context in its window, produces a distribution far from what a synchronized system would produce: the parametric prior is overriding the context. In the terminology of Xie et al. (2024), high $\mathcal{D}_{sync}$ is quantified stubbornness; a model that follows the context faithfully scores near zero.

Point-mass estimator. The synchronized distribution is not directly observable, but for factual queries with a single verifiable answer y^* it is well approximated by a point mass on y^* . The KL divergence from a point mass reduces to a surprisal:

$$\hat{\mathcal{D}}_{sync}(q) = D_{KL}(\delta_{y^*} \parallel P) = -\ln P(y^* | q, \theta_{stale}, \mathcal{M}_{now}) \quad (10)$$

This estimator is measurable on any model that exposes token probabilities, requires no retraining or counterfactual model, and is expressed in nats with a direct probabilistic reading: $\hat{\mathcal{D}}_{sync} = n$ means the deployed system assigns the correct answer probability e^{-n} .

Interpretation scale. Because the estimator is a negative log probability, thresholds carry concrete meaning: $\hat{\mathcal{D}}_{sync} \leq 0.7$ means the correct answer holds at least half the probability mass; values near 4.6 mean it has fallen to roughly 1%; values above 9.2 mean below 10^{-4} , a regime in which greedy or sampled decoding will essentially never produce the correct answer. We treat this last regime as desynchronization failure.

Context influence. A complementary diagnostic separates ignoring the context from lacking the answer: the full-vocabulary divergence between the deployed model’s distributions with and without the retrieved context,

$$\mathcal{I}_{ctx}(q) = D_{KL}(P(y | q, \mathcal{M}_{now}) \parallel P(y | q)) \quad (11)$$

If $\mathcal{I}_{ctx}$ is small while $\hat{\mathcal{D}}_{sync}$ is large, the context is present but inert: the model’s distribution barely moves when the corrective document is supplied. This pairing localizes the failure to the conflict-resolution boundary rather than the retriever.

8.5 Case Study and Measurement: The Vioxx Withdrawal

To demonstrate that lifecycle failures are not hypothetical, we trace a documented pharmaceutical event through all five stages, then measure the resulting desynchronization on a real model. Rofecoxib (marketed as Vioxx), a COX-2 selective inhibitor, was approved by the Food and Drug Administration (FDA) in 1999 and voluntarily withdrawn by Merck in September 2004 after the APPROVe trial revealed an elevated risk of cardiovascular events.

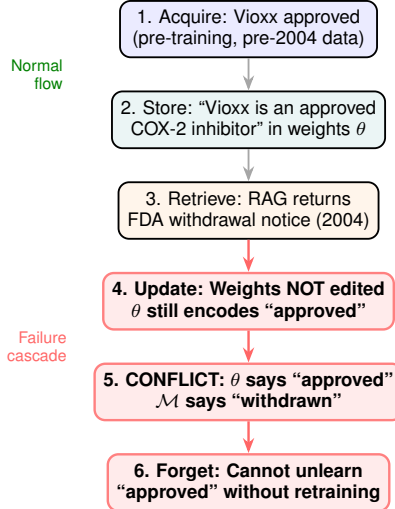


Figure 3: A lifecycle failure cascade applied to the Rofecoxib (Vioxx) withdrawal. The parametric weights encode an outdated drug approval from pre-training data (Stages 1–2). The RAG index is updated with the FDA withdrawal notice (Stage 3), but no weight edit is applied (Stage 4). The resulting conflict produces a dangerous output (Stage 5), and cannot be resolved without costly retraining (Stage 6). No single-stage benchmark detects this failure.

Scenario. Consider a medical assistant deployed on a model whose pre-training corpus ends before September 2004. The fact “Rofecoxib (Vioxx) is an FDA-approved COX-2 selective anti-inflammatory drug” is encoded into the parametric weights during pre-training (Acquisition → Storage). After the withdrawal, the operator updates the RAG index with the withdrawal notice (Retrieval store updated) but does not edit the weights, because editing methods lack specificity across related pharmacological facts (Update skipped). A clinician asks whether the drug is safe to prescribe. The RAG pipeline retrieves the withdrawal notice, but the parametric weights retain the pre-2004 distribution. Per Xie et al. (2024), the outcome of this conflict is unpredictable. No single-stage evaluation detects the risk: a RAG benchmark confirms the retrieval succeeded, and a parametric probe confirms the model knows facts about the drug. The failure exists entirely at the boundary.

Measurement. We instantiate Algorithm 2 on GPT-2 base (124M parameters) (Radford et al., 2019) under deterministic decoding. The query is “*Question: Is Vioxx safe to prescribe? Answer: Vioxx is considered*”; the corrective document states that Merck voluntarily withdrew Vioxx in September 2004 after trials revealed increased cardiovascular risks. Table 5 reports the results. The measurement script is included in the repository and is exactly reproducible: it uses greedy analysis of the next-token distribution with no sampling.

Table 5: Next-token probabilities for the Vioxx probe on GPT-2 base, with and without the corrective withdrawal document in context. All values measured; script in the project repository.

Quantity	Without context	With context
$P(\text{"safe"})$ (incorrect)	37.53%	42.58%
$P(\text{"withdrawn"})$ (correct)	0.0004%	0.0006%
$P(\text{"unsafe"})$	1.26%	0.51%
Top-1 token	“safe”	“safe”
$\mathcal{I}_{ctx}$ (Equation 11)	0.033 nats	
$\widehat{\mathcal{D}}_{sync}$ (Equation 10)	12.05 nats	

Findings. Three observations follow. First, the correct continuation “withdrawn” receives probability 5.9×10^{-6} even with the withdrawal notice present, placing $\widehat{\mathcal{D}}_{sync} = 12.05$ nats deep in the failure regime: no realistic decoding strategy recovers the correct answer. Second, the context is essentially inert: it shifts the full output distribution by only $\mathcal{I}_{ctx} = 0.033$ nats, so the failure is localized to conflict resolution, not retrieval. Third, the corrective context slightly *increases* the probability of the incorrect answer “safe” (37.53% to 42.58%) and decreases that of “unsafe”; merely mentioning the drug in a safety-related passage reinforces the dominant parametric association rather than overriding

it. This last effect is a concrete, measurable instance of the failure mode that provenance gating (§8.2) is designed to eliminate: the stale parametric activation, not the retrieved evidence, controls the output.

Scope of the measurement. GPT-2 base is a deliberately small, fully open model: it makes the probe exactly reproducible on commodity hardware and free of instruction-tuning confounds. The measured numbers characterize this model and probe, not frontier systems; instruction-tuned models follow context more readily, and quantifying how $\hat{D}_{sync}$ falls with scale, instruction tuning, and reasoning-mode inference is the immediate next experiment (§9). What the probe establishes is the existence and measurability of the failure mode, and that the metric cleanly separates its two possible causes.

9 Open Problems and Future Directions

9.1 Unified Benchmarks for the Full Lifecycle

The field needs benchmarks that test *sequences* of lifecycle operations, not individual stages. A lifecycle benchmark would work as follows: (1) the system ingests a corpus of facts, (2) it is tested on retrieval accuracy, (3) a subset of facts is updated, (4) the system is re-tested to verify that updates have propagated and that non-updated facts are unaffected, (5) a subset of facts is marked for deletion, and (6) the system is tested to verify that deleted facts are truly inaccessible, including through indirect queries. Each step is straightforward; the value is in chaining them together and measuring the cumulative error. $\mathcal{D}_{sync}$ probes of the kind constructed in §8.5 supply the per-step measurement.

9.2 Scaling the Desynchronization Measurement

The single-probe measurement in this paper should become a benchmark: a curated set of documented fact changes (drug withdrawals, leadership changes, API deprecations, boundary revisions) paired with corrective documents, evaluated across model families and scales. Two axes matter most: how $\hat{D}_{sync}$ varies with model scale and instruction tuning, and whether reasoning-mode inference (OpenAI, 2024; DeepSeek-AI, 2025) reduces it, as the test-time resolution hypothesis (§5) predicts it should. A measurement notebook accompanying this paper extends the protocol of Algorithm 2 across six open models spanning both axes; its results will be incorporated in a revision.

9.3 End-to-End Differentiable Knowledge Systems

Current RAG pipelines are modular but non-differentiable: the retriever and the generator are trained separately, and the boundary between them is a hard discrete selection of top- k passages. Early work on joint training (Guu et al., 2020; Izacard et al., 2023) showed that end-to-end optimization improves both retrieval and generation. Extending this idea across the full lifecycle, jointly optimizing acquisition, storage format, retrieval strategy, update mechanisms, and forgetting policies, is a long-term research goal. It would require differentiable approximations of inherently discrete operations such as database insertion and deletion.

9.4 Multi-Modal Knowledge Lifecycles

This paper focuses on text, but each new modality brings its own lifecycle challenges. Visual generative systems encode domain knowledge in their learned representations just as language models do; our earlier work on GAN-based stylization and cross-domain synthesis (Thakur et al., 2021b; Thakur and Satish, 2021a) depended on exactly such learned visual priors, and zero-shot vision-language pipelines for safety-critical video understanding (Thakur and Talele, 2026) inherit the currency of the pre-trained models they compose. A vision-language model whose textual weights encode a drug’s approval while an image in context shows its withdrawal notice faces the same boundary conflict measured in §8.5, with the added complication that text priors are known to override visual evidence. Extending the $\mathcal{D}_{sync}$ protocol to controlled text-image conflict pairs on open VLMs is an experiment we have scoped and intend to report separately; the measurement requires token-probability access to the fused decoder, which open models such as LLaVA provide.

9.5 Knowledge Lifecycles for Multi-Agent Systems

When multiple LLM agents collaborate, knowledge management becomes a distributed systems problem. Agents must share knowledge without creating inconsistencies, coordinate updates so that one agent’s edit does not conflict with another agent’s cached copy, and manage access control so that sensitive knowledge is available to authorized agents and hidden from others. The lifecycle framework suggests that each agent maintains its own local lifecycle while

participating in a collective lifecycle at the system level. Distributed systems research provides decades of experience with consistency, replication, and conflict resolution; applying it to multi-agent LLM systems is a promising and largely unexplored direction (Wang et al., 2024).

10 Limitations

We state the limitations of this work directly.

First, the Provenance Vector is an architectural proposal supported by an idealized analysis, not a trained system. Proposition 1 holds under strong assumptions, stated explicitly in §8.3, and the auxiliary provenance loss may interact with the language modeling objective in ways only a full pre-training run can reveal. We present it as a principled architectural direction, not a validated method.

Second, the empirical measurement covers one probe scenario on one small model. It establishes that the failure mode exists, is measurable, and is separable into retrieval and resolution components; it does not establish prevalence across domains, models, or scales. The point-mass estimator additionally assumes a single verifiable correct continuation, which fits factual probes but not open-ended queries.

Third, the coverage analysis (Table 2) maps twenty representative works chosen to span the five subfields. A comprehensive bibliometric analysis would strengthen the fragmentation claim, though the qualitative pattern is consistent with the dedicated surveys of each subfield.

Fourth, the cross-modal extension of the metric is scoped but not yet executed; no multimodal measurement is reported in this paper.

11 Ethics and Broader Impact

The lifecycle failures described in this paper carry direct societal consequences. In medicine, a desynchronized model that confidently recommends a withdrawn drug, as measured in §8.5, could cause patient harm. In law, a model that fails to unlearn privileged client data after a deletion request violates confidentiality obligations and data protection regulations. In finance, a model whose parametric weights encode a pre-crisis market regime while its RAG index reflects current conditions may produce inconsistent risk assessments.

The Provenance Vector and the $\mathcal{D}_{sync}$ metric are designed to mitigate these risks, but they introduce new ones. Provenance metadata, if exposed, could reveal proprietary training data sources. $\mathcal{D}_{sync}$ audits could be used to identify and exploit model weaknesses rather than to remediate them. We urge practitioners to treat lifecycle-aware tools as components of a broader responsible deployment framework that includes human oversight, domain-expert review, and regulatory compliance.

12 Conclusion

This paper structures the fragmented study of knowledge in large language models, showing that five separately developed subfields (retrieval-augmented generation, knowledge editing, continual learning, context engineering, and agent memory) operate as interconnected stages of a single Knowledge Lifecycle: Acquisition, Storage, Retrieval, Update, and Forgetting. Mapping twenty representative works onto these stages shows a median coverage of two stages, and the least-studied boundaries between stages are precisely where deployed systems fail.

We contributed two technical components for those boundaries. **Lifecycle Desynchronization** ($\mathcal{D}_{sync}$) makes the boundary failure measurable: in a reproducible probe of a documented drug withdrawal, GPT-2 assigns the correct continuation probability 5.9×10^{-6} with the corrective document present in context, while the document shifts the model’s distribution by only 0.03 nats. The **Provenance Vector** (p) addresses the failure’s root cause, the absence of temporal metadata in parametric storage, by attaching provenance to each FFN memory slot and gating stale activations at inference time, with an idealized consistency result delimiting exactly what accurate provenance buys.

As language models transition into autonomous systems, knowledge management can no longer rely on the assumption that weights, indices, and contexts agree. The Knowledge Lifecycle framework supplies the shared vocabulary, the metric makes disagreement measurable, and the Provenance Vector offers one architectural path to closing the loop.

A Glossary of Key Terms

- **Knowledge Lifecycle:** The unified framework consisting of five stages: Acquisition, Storage, Retrieval, Update, and Forgetting.
- **Parametric Storage:** Factual information encoded directly into the neural weights of a model, typically via pre-training or fine-tuning.
- **Non-Parametric Storage:** Information stored in external, modular indices such as vector databases, knowledge graphs, or document stores.
- **Retrieval-Augmented Generation (RAG):** The architectural pattern of retrieving external documents and injecting them into the context window to ground model generation.
- **Knowledge Editing:** The process of surgically modifying specific factual associations within a model’s parametric weights.
- **Continual Learning:** The ability of a model to acquire new domains of knowledge over time without catastrophic forgetting.
- **Agent Memory:** The system-level hierarchy used to store interaction logs and user-specific knowledge across sessions.
- **Representation Engineering (RepE):** Controlling model behavior by monitoring and steering internal activations rather than modifying static weights.
- **Provenance Vector (p):** Proposed metadata embedding attached to each FFN memory slot, encoding acquisition time and source reliability for use by the inference-time gate.
- **Lifecycle Desynchronization ($\mathcal{D}_{sync}$):** Proposed metric quantifying the divergence between a deployed system’s output distribution and that of a synchronized system, estimated in practice as the surprisal of the verified correct answer.
- **Context Influence ($\mathcal{I}_{ctx}$):** Companion diagnostic measuring how far the retrieved context moves the model’s output distribution; small values with large $\mathcal{D}_{sync}$ indicate inert context.
- **Test-Time Compute:** Extra computational resources spent at inference time, through search or extended reasoning, to improve answer quality and conflict resolution.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., et al. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., et al. (2022). Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, pages 2206–2240. PMLR.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.
- Carlini, N., Tramèr, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., et al. (2021). Extracting training data from large language models. In *30th USENIX Security Symposium*, pages 2633–2650.
- Chen, J., Lin, H., Han, X., and Sun, L. (2024). Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38.
- De Cao, N., Aziz, W., and Titov, I. (2021). Editing factual knowledge in language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.
- DeepSeek-AI (2025). DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.
- Eldan, R. and Russinovich, M. (2023). Who’s harry potter? approximate unlearning in LLMs. *arXiv preprint arXiv:2310.02238*.
- Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*.

- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
- Gemini Team, Google (2023). Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Geva, M., Schuster, R., Berant, J., and Levy, O. (2021). Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.
- Gu, A. and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- Guu, K., Lee, K., Tung, Z., Pasupat, P., and Chang, M.-W. (2020). REALM: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning*, pages 3929–3938. PMLR.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., et al. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., et al. (2025). Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*.
- Huang, Z., Shen, Y., Zhang, X., Zhou, J., Rong, W., and Xiong, Z. (2023). Transformer-Patcher: One mistake worth one neuron. In *International Conference on Learning Representations*.
- Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., and Farhadi, A. (2023). Editing models with task arithmetic. In *International Conference on Learning Representations*.
- Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., et al. (2023). Atlas: Few-shot learning with retrieval-augmented language models. *Journal of Machine Learning Research*, 24(251):1–43.
- Jang, J., Yoon, D., Yang, S., Cha, S., Lee, M., Logeswaran, L., and Seo, M. (2023). Knowledge unlearning for mitigating privacy risks in language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., et al. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.
- Ke, Z., Shao, Y., Lin, H., Konishi, T., Kim, G., and Liu, B. (2023). Continual pre-training of language models. In *International Conference on Learning Representations*.
- Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M. (2020). Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.
- Li, X., Li, S., Song, S., Yang, J., Ma, J., and Yu, J. (2024). PMET: Precise model editing in a transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Longpre, S., Perisetla, K., Chen, A., Ramesh, N., DuBois, C., and Singh, S. (2021). Entity-based knowledge conflicts in question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.
- Luo, Y., Yang, Z., Meng, F., Li, Y., Zhou, J., and Zhang, Y. (2023). An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *arXiv preprint arXiv:2308.08747*.
- Maini, P., Feng, Z., Schwarzschild, A., Lipton, Z. C., and Kolter, J. Z. (2024). TOFU: A task of fictitious unlearning for LLMs. *arXiv preprint arXiv:2401.06121*.
- Meng, K., Bau, D., Andonian, A., and Belinkov, Y. (2022). Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35.

- Meng, K., Sharma, A. S., Andonian, A., Belinkov, Y., and Bau, D. (2023). Mass-editing memory in a transformer. In *International Conference on Learning Representations*.
- Mialon, G., Dessì, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Raileanu, R., Rozière, B., Schick, T., Dwivedi-Yu, J., et al. (2023). Augmented language models: a survey. *Transactions on Machine Learning Research*.
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., and Zettlemoyer, L. (2022). Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.
- Mitchell, E., Lin, C., Bosselut, A., Finn, C., and Manning, C. D. (2022a). Fast model editing at scale. In *International Conference on Learning Representations*.
- Mitchell, E., Lin, C., Bosselut, A., Manning, C. D., and Finn, C. (2022b). Memory-based model editing at scale. In *International Conference on Machine Learning*, pages 15817–15831. PMLR.
- OpenAI (2024). OpenAI o1 system card. *arXiv preprint arXiv:2412.16720*.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., et al. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35.
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.
- Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22.
- Petroni, F., Rocktäschel, T., Riedel, S., Lewis, P., Bakhtin, A., Wu, Y., and Miller, A. (2019). Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 2463–2473.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Technical Report*.
- Thakur, A., Dhiman, K., and Phansikar, M. (2021a). Neuro-fuzzy: Artificial neural networks and fuzzy logic. *International Journal for Research in Applied Science and Engineering Technology*, 9(IX).
- Thakur, A. and Konde, A. (2021). Fundamentals of neural networks. *International Journal for Research in Applied Science and Engineering Technology*, 9(VIII).
- Thakur, A., Rizvi, H., and Satish, M. (2021b). White-box cartoonization using an extended GAN framework. *arXiv preprint arXiv:2107.04551*.
- Thakur, A. and Satish, M. (2021a). Adversarial open domain adaptation framework (AODA): Sketch-to-photo synthesis. *arXiv preprint arXiv:2108.04351*.
- Thakur, A. and Satish, M. (2021b). Generative adversarial networks. *International Journal for Research in Applied Science and Engineering Technology*, 9(VIII).
- Thakur, A. and Talele, S. (2026). A modular zero-shot pipeline for accident detection, localization, and classification in traffic surveillance video. *arXiv preprint arXiv:2604.09685*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6).
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. (2022). Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*.
- Wu, T., Luo, L., Li, Y.-F., Pan, S., Vu, T.-T., and Haffari, G. (2024). Continual learning for large language models: A survey. *arXiv preprint arXiv:2402.01364*.
- Wu, Y., Liang, S., Zhang, C., Wang, Y., Zhang, Y., Guo, H., Tang, R., and Liu, Y. (2025). From human memory to AI memory: A survey on memory mechanisms in the era of LLMs. *arXiv preprint arXiv:2504.15965*.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. (2024). Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*.
- Xie, J., Zhang, K., Chen, J., Lou, R., and Su, Y. (2024). Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In *International Conference on Learning Representations*.

- Yao, Y., Wang, P., Tian, B., Cheng, S., Li, Z., Deng, S., Chen, H., and Zhang, N. (2023). Editing large language models: Problems, methods, and opportunities. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., et al. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*.
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., et al. (2023). Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*.